

1. Описание условия задания

Цель работы: реализовать алгоритм обработки графовых структур: проверка связности. Обработать графовую структуру в соответствии с указанным вариантом задания: определить, является ли связным заданный граф. Обосновать выбор необходимого алгоритма и выбор структуры для представления графов. Предложить вариант реальной задачи, для решения которой можно использовать разработанную программу. Ввод данных – на усмотрение программиста. Результат выдать в графической форме.

2. Техническое задание

Внешняя спецификация программы:

Исходные данные:

- Набор узлов графа и связей между ними.
- Выбор операции через консольное меню.

Выходные данные:

- Изображение графа.
- Результат операций с графом.
- Статистика затраченного времени выполнения и объема памяти при проверке графа на связность.

Ограничения:

- Каждые два узла может соединять максимум один путь.

Способ обращения: консольный ввод/вывод.

Команда запуска программы: ./app.exe. Дальнейшее взаимодействие осуществляется с помощью меню.

Аварийные ситуации:

- “ERROR_MEM” – Ошибка выделения памяти.
- “ERROR_IO” – Ошибка ввода.

Возможные действия, реализуемой программой:

1. Ввести острова и пути между ними.
2. Добавить остров.
3. Добавить путь между существующими островами.
4. Проверить связность всех островов.
5. Вывести все пути между островами.
6. Собрать статистику.
7. Очистить все пути и острова.
8. 0 - Выход.

3. Внутренние структуры данных

В рамках выполнения данной лабораторной работы мной были реализованы следующие структуры данных:

Узел списка смежности

<pre>typedef struct adj_node { int vertex; struct adj_node *next; } adj_node_t;</pre>

- **vertex** – номер смежной вершины (острова), с которой соединён текущий узел.
- **next** – указатель на следующий узел в списке смежности данной вершины.

Граф: список смежности

<pre>typedef struct graph { int num_vertices; adj_node_t **adj_list; } graph_t;</pre>

- **num_vertices** – количество вершин в графе.
- **adj_list** – динамический массив указателей, где каждый элемент `adj_list[i]` указывает на начало списка смежности для вершины с индексом `i`.

4. Описание Алгоритмов

Основной алгоритм программы:

1. Вывести меню;
2. Считать выбор пользователя и проверить его корректность;
3. Выполнить соответствующее действие;
4. Либо снова вывести меню с ожиданием выбора, либо завершить программу с сообщением об ошибке.

Алгоритм добавления пути между вершинами графа

1. Если указатель на граф `g` равен `NULL` ИЛИ хотя бы один из индексов вершин `u` или `v` выходит за допустимые границы `[0, g->num_vertices - 1]`:
 - 1.1. Прервать выполнение — добавление невозможно.
2. Иначе:
 - 2.1. Начать обход списка смежности вершины `u`.
 - 2.2. Проверить, существует ли в этом списке узел с номером вершины `v`.
 - 2.3. Если такой узел найден И флаг `quiet == 0`:
 - 2.3.1. Вывести сообщение: «Такой путь уже существует.»
 - 2.3.2. Прервать выполнение — дубликат не добавляется.
3. Создать новый узел списка смежности для вершины `v`.
4. Добавить этот узел в начало списка смежности вершины `u`.
5. Если `u ≠ v` (то есть ребро — не петля):
 - 5.1. Создать новый узел для вершины `u`.
 - 5.2. Добавить его в начало списка смежности вершины `v`.
6. Если флаг `quiet == 0`:
 - 6.1. Вывести сообщение: «Путь между островами `u+1` и `v+1` добавлен.»
7. Вернуть 1 — путь успешно добавлен.

Алгоритм добавления новой вершины

1. Если указатель на указатель графа `g` равен `NULL`:
 - 1.1. Вернуть код ошибки `ERROR_IO`.
2. Если сам граф `*g` не инициализирован (`*g == NULL`):
 - 2.1. Создать новый граф с одной вершиной (`graph_create(1)`).
 - 2.2. Если создание не удалось — вернуть `ERROR_MEM`.
 - 2.3. Иначе — вернуть `ERROR_OK`.
3. Иначе (`*g` уже существует):
 - 3.1. Сохранить указатель на текущий граф (`old_g = *g`).
 - 3.2. Определить текущее число вершин (`old_n = old_g->num_vertices`).
 - 3.3. Увеличить количество вершин на 1 (`new_n = old_n + 1`).

- 3.4.Расширить массив указателей на списки смежности с помощью realloc до размера new_n.
- 3.5.Если realloc вернул NULL:
 - 3.5.1. Вернуть ERROR_MEM (нехватка памяти).
- 3.6.Иначе:
 - 3.6.1. Присвоить новую память массиву adj_list.
 - 3.6.2. Установить adj_list[old_n] = NULL (новая вершина изначально изолирована).
 - 3.6.3. Обновить количество вершин: old_g->num_vertices = new_n.
 - 3.6.4. Вернуть ERROR_OK.

Алгоритм проверки связности графа

- 1. Если граф g равен NULL ИЛИ количество вершин равно 0:
 - 1.1.Считать граф связным и вернуть 1 (по соглашению для пустого графа).
- 2. Если в графе ровно одна вершина:
 - 2.1.Вернуть 1 — граф тривиально связан.
- 3. Иначе:
 - 3.1.Выделить массив visited размером g->num_vertices, инициализированный нулями.
 - 3.2.Если выделение не удалось — вернуть 0 (ошибка памяти).
- 4. Выполнить поиск в глубину (DFS), начиная с вершины 0:
 - 4.1.Отметить текущую вершину как посещённую (visited[v] = 1).
 - 4.2.Для каждой вершины w, смежной с текущей:
 - 4.2.1. Если w ещё не посещена (visited[w] == 0):
 - 4.2.1.1. Рекурсивно запустить DFS из w.
- 5. После завершения обхода:
 - 5.1.Пройти по всем элементам массива visited.
 - 5.2.Если хотя бы один элемент равен 0:
 - 5.2.1. Освободить память массива visited.
 - 5.2.2. Вернуть 0 — граф несвязен (существует изолированная компонента).
- 6. Если все элементы visited[i] == 1:
 - 6.1.Освободить память массива visited.
 - 6.2.Вернуть 1 — граф связан.

Алгоритм использует поиск в глубину (DFS) как основной метод обхода. Сложность алгоритма — $O(n + m)$ по времени и $O(n)$ по памяти, где n — число вершин, m — число рёбер.

Реализуемые функции:

Реализуемые функции:

// Функция очистки буфера ввода после scanf

1. void clear_input_buffer(void);

// Функция открытия изображения с помощью системной команды

2. void show_image(const char *image_path);

// Функция вывода сообщения об ошибке по её коду

3. void print_error(int rc);

// Функция вывода главного меню программы

4. void print_menu(void);

// Функция создания графа с заданным количеством вершин

5. graph_t *graph_create(int n);

// Функция добавления ребра между двумя вершинами (с опциональным подавлением вывода)

6. int graph_add_edge(graph_t *g, int u, int v, int quiet);

// Функция динамического добавления новой изолированной вершины в граф

7. int graph_add_vertex(graph_t **g);

// Функция проверки связности неориентированного графа с помощью DFS

8. int graph_is_connected(graph_t *g);

// Функция освобождения всей памяти, занятой графом

9. void graph_free(graph_t *g);

// Функция экспорта графа в DOT-файл для визуализации

10. int graph_create_dot(graph_t *g, const char *filename);

// Функция генерации случайного неориентированного графа заданного размера

11. graph_t *graph_generate_random(int n);

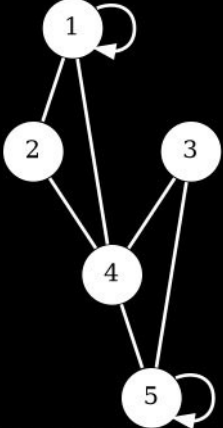
// Функция расчёта объёма памяти, занимаемой графом (в байтах)

12. size_t graph_memory_usage(const graph_t *g);

// Функция выполнения замеров времени и памяти для алгоритма проверки связности

13. int statistic(void);

5. Тесты

Позитивные тесты:	
Действие	Результат
Вывод пустого графа	“Граф пустой”
Вывод графа из одного узла	
Вывод графа из нескольких узлов	
Инициализация графа	Введите начальное количество островов: 5 Введите количество путей: 5 Путь 1: введите два острова (от 1 до 5): 1 1 Путь между островами 1 и 1 добавлен. Путь 2: введите два острова (от 1 до 5): 1 2 Путь между островами 1 и 2 добавлен. Путь 3: введите два острова (от 1 до 5): 3 4 Путь между островами 3 и 4 добавлен. Путь 4: введите два острова (от 1 до 5): 4 5 Путь между островами 4 и 5 добавлен. Путь 5: введите два острова (от 1 до 5): 5 5 Путь между островами 5 и 5 добавлен. Карта из 5 островов и 5 путей создана.
Добавление узла графа	
Добавление существующего пути	
Добавление нового пути	
Проверка графа на связность	
Очистка графа	
Замер времени и затраченной памяти.	

Негативные тесты:

Ввод невалидного значения при выборе меню.

6. Оценка эффективности

Замер времени и памяти алгоритма проверки связности

Кол-во вершин	Время (сек)	Память (байт)
50	0.000018	20000
100	0.000095	80016
500	0.001990	2000016
1000	0.024449	8000016

Наблюдения результатов по эффективности:

1. Время выполнения растёт нелинейно с увеличением числа вершин.
 - 1.1. При увеличении количества вершин с 50 до 1000 (в 20 раз) время выполнения возрастает с 0.000018 с до 0.024449 с — более чем в 1300 раз.
 - 1.2. Это связано с ростом не только вершин, но и рёбер: в генерируемом графе плотность фиксирована (~50% от максимального числа рёбер), поэтому количество рёбер растёт квадратично ($m \approx O(n^2)$).
 - 1.3. Следовательно, общая сложность алгоритма $O(n + m) \approx O(n^2)$ в случае плотных графов, что и наблюдается на практике.
2. Объём памяти растёт квадратично от числа вершин.
 - 2.1. При $n = 50$ — 20 000 байт, при $n = 100$ — 80 016 байт (рост в ~4 раза при удвоении n).
3. Алгоритм DFS эффективно использует память при обходе.
 - 3.1. Несмотря на рекурсивную реализацию, стек вызовов глубиной не превышает n , а в разреженных графах — значительно меньше.
 - 3.2. Основной объём памяти потребляется хранением структуры графа, а не самим алгоритмом проверки связности — что подтверждает правильность выбора представления.

Обоснование выбора алгоритма и структуры данных:

1. Почему выбран список смежности?
 - 1.1. Экономия памяти в разреженных графах. Матрица смежности требует $O(n^2)$ памяти независимо от плотности, тогда как список смежности — $O(n + m)$.
 - 1.2. Гибкость при динамическом изменении. В ходе работы программы пользователь может добавлять вершины и рёбра. Список смежности легко расширяется, тогда как матрицу пришлось бы полностью пересоздавать.
2. Почему использован поиск в глубину (DFS), а не поиск в ширину (BFS)?

- 2.1. Цель задачи — проверка связности, а не поиск кратчайшего пути. Оба алгоритма (DFS и BFS) корректно определяют, достижимы ли все вершины из начальной, и оба имеют линейную сложность $O(n + m)$.
- 2.2. Простота реализации. Рекурсивный DFS требует меньше кода и не нуждается в явной очереди (в отличие от BFS, который требует структуры данных для хранения уровня обхода).
- 2.3. Меньшее потребление памяти в среднем случае. Хотя в худшем случае (вырожденный граф-цепочка) стек рекурсии DFS может достигать $O(n)$, в сбалансированных или разреженных графах глубина стека обычно меньше, чем размер очереди в BFS.

7. Выводы

Список смежности оказался оптимальным представлением графа благодаря своей адаптивности к разреженным структурам и линейной зависимости объёма памяти от числа рёбер ($O(n + m)$). В отличие от матрицы смежности, он позволяет эффективно масштабировать граф динамически — добавлять вершины и рёбра без полного пересоздания структуры.

Алгоритм поиска в глубину (DFS) оказался целесообразным выбором для проверки связности: он прост в реализации, требует минимальных накладных расходов и корректно решает поставленную задачу за линейное время $O(n + m)$.

Результаты замеров подтвердили теоретические оценки: при увеличении размера графа время выполнения и потребление памяти растут в соответствии со сложностью алгоритма.

Решение реальной задачи:

Определение связности островов. Возможно ли из первого острова добраться до всех остальных с помощью морских путей.

8. Ответы на контрольные вопросы

1. Что такое граф?

- Граф — это конечное множество вершин V и рёбер E , соединяющих эти вершины: $G = \langle V, E \rangle$.
- Если рёбра имеют направление, граф называется ориентированным (орграф).
- Если направления нет — неориентированным (неорграф).
- Если рёбра имеют вес (стоимость), граф называется взвешенным.
- Связным называется неориентированный граф, в котором существует путь между любой парой вершин.

2. Как представляются графы в памяти?

Основные способы представления графов в памяти:

- Матрица смежности — квадратная матрица размером $(n * n)$, где элемент $(b[i][j] = 1)$, если существует ребро между вершинами (i) и (j) , и (0) — иначе. У неорграфа матрица симметрична. Требует $(O(n^2))$ памяти.
- Список смежности — для каждой вершины хранится список (или связный список) смежных с ней вершин. Требует $(O(n + m))$ памяти и предпочтителен для разреженных графов.

3. Какие операции возможны над графами?

Среди основных операций над графами:

- Поиск кратчайшего пути от одной вершины к другой.
- Поиск кратчайших путей от одной вершины ко всем остальным (алгоритмы Дейкстры, Беллмана–Форда).
- Поиск кратчайших путей между всеми парами вершин (алгоритм Флойда–Уоршалла).
- Поиск эйлера пути (проходящего по каждому ребру ровно один раз).
- Поиск гамильтонова пути (проходящего по каждой вершине ровно один раз).
- Проверка связности графа.
- Построение остовного дерева минимальной стоимости (алгоритмы Крускала и Прима).

4. Какие способы обхода графов существуют?

Два основных способа систематического обхода графа:

- Поиск в глубину (DFS, Depth-First Search) — рекурсивный обход: из текущей вершины сразу углубляется в одного из соседей, обрабатывая граф «вглубь». Используется для проверки связности, построения остовов, топологической сортировки и др.
- Поиск в ширину (BFS, Breadth-First Search) — итеративный обход с использованием очереди: сначала обрабатываются все соседи текущей вершины, затем их соседи и т.д. Гарантирует нахождение кратчайшего пути в невзвешенном графе.

5. Где используются графовые структуры?

Графы применяются в самых разных областях:

- Компьютерные сети — моделирование узлов и соединений (маршрутизация, проверка связности).
- Транспортные системы — поиск кратчайших маршрутов, планирование логистики.
- Социальные сети — анализ связей между пользователями.
- Электронные схемы — представление компонентов и проводников.
- Компиляторы — зависимости между модулями, оптимизация кода.

6. Какие пути в графе Вы знаете?

- Простой путь — путь, в котором все вершины различны.
- Эйлеров путь — путь, проходящий по каждому ребру ровно один раз. Существует в связном графе, если число вершин нечётной степени ≤ 2 .
- Эйлеров цикл — замкнутый эйлеров путь. Существует, если все вершины имеют чётную степень.
- Гамильтонов путь — путь, проходящий по каждой вершине ровно один раз.
- Гамильтонов цикл — замкнутый гамильтонов путь.

7. Что такое каркасы графа?

Каркас (или остов, стягивающее дерево) — это ациклический связный подграф, содержащий все вершины исходного графа и некоторое подмножество его рёбер.

- В остове любого связного графа с (n) вершинами ровно $(n - 1)$ рёбер.
- Если граф взвешенный, можно строить остов минимальной стоимости, используя алгоритмы Крускала или Прима.